

Gradient Descent

Optimization

Strategy #1: A first very bad idea solution: Random search

Optimization

Strategy #1: A first very bad idea solution: Random search

```
% assume X_train is the data where each row is an example
% assume Y_train are the labels
% assume the function L evaluates the loss function

bestloss = Inf; % the highest possible value
for num=1:1000
    W = randn(D+1,1)*0.001; % generate random parameters
    loss = L(X_train, Y_train, W); % get the loss over the entire training set
    if (loss < bestloss) % keep track of the best solution
        bestloss = loss;
        bestW = W;
    end
    ['in attempt ' num2str(num) ' the loss was ' num2str(loss) ', best ' num2str(bestloss)]
end
```

Optimization

Let's see how well this works on the test set...

```
%accuracy on the test set  
correct=sum(Y_test == (sigmoid(X_test*bestW) > 0.5));  
accuracy_test = correct / N_test;
```

Optimization

Strategy #2: Random Local Search

```
bestloss = Inf; % the highest possible value
W = randn(D+1,1)*0.001; % generate random starting W
step_size = 0.0001;
for i=1:1000
    Wtry = W + randn(D+1,1) * step_size;
    loss = L(X_train,Y_train, Wtry);
    if (loss < bestloss)
        W = Wtry;
        bestloss = loss;
    end
    ['iter ' num2str(i) ' the loss was ' num2str(bestloss)]
end
```

Optimization

Strategy #3: Follow the slope



A person is stuck in the mountains and is trying to get down. There is heavy fog such that visibility is extremely low.

Source: https://en.wikipedia.org/wiki/Gradient_descent

Source: Fei-Fei Li, Ranjay Krishna, Danfei Xu, CS231n: Convolutional Neural Networks for Visual Recognition

Optimization

Strategy #3: Follow the slope



A person is stuck in the mountains and is trying to get down. There is heavy fog such that visibility is extremely low.

The path down the mountain is not visible, so they must use local information to find the minimum.

Optimization

Strategy #3: Follow the slope



A person is stuck in the mountains and is trying to get down. There is heavy fog such that visibility is extremely low.

The path down the mountain is not visible, so they must use local information to find the minimum.

It can use the method of gradient descent, which involves looking at the steepness of the hill at their current position, then proceeding in the direction with the steepest descent (i.e. downhill).

Optimization

Strategy #3: Follow the slope



A person is stuck in the mountains and is trying to get down. There is heavy fog such that visibility is extremely low.

The path down the mountain is not visible, so they must use local information to find the minimum.

It can use the method of gradient descent, which involves looking at the steepness of the hill at their current position, then proceeding in the direction with the steepest descent (i.e. downhill).

Using this method, they would eventually find their way down the mountain or possibly get stuck in some hole.

Source: https://en.wikipedia.org/wiki/Gradient_descent

Source: Fei-Fei Li, Ranjay Krishna, Danfei Xu, CS231n: Convolutional Neural Networks for Visual Recognition

Optimization

In 1-dimension, the derivative of a function:

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Optimization

In 1-dimension, the derivative of a function:

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

In multiple dimensions, the **gradient** is the vector of (partial derivatives) along each dimension.

Optimization

In 1-dimension, the derivative of a function:

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

In multiple dimensions, the **gradient** is the vector of (partial derivatives) along each dimension.

The slope in any direction is the **dot product** of the direction with the gradient.

Optimization

In 1-dimension, the derivative of a function:

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

In multiple dimensions, the **gradient** is the vector of (partial derivatives) along each dimension.

The slope in any direction is the **dot product** of the direction with the gradient.

The direction of steepest descent is the **negative gradient**.

Optimization

Current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

Optimization

Current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W+h: (first dim)

[0.34+0.0001,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25322

Optimization

Current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W+h: (first dim)

[0.34+0.0001,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25322

Gradient dW

[-2.5,
?,
?,
?,
?,
?,
?,
?,
?,
?,...]


$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

(1.25322 - 1.25347)/0.0001
= -2.5

Optimization

Current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W+h: (second dim)

[0.34,
-1.11+0.0001,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25352

Gradient dW

[-2.5,
0.6,
?,
?,
?,
?,
?,
?,
?,
?,...]

$(1.25353 - 1.25347)/0.0001$
 $= 0.6$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Optimization

Current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W+h: (third dim)

[0.34,
-1.11,
0.78+0.0001,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

Gradient dW

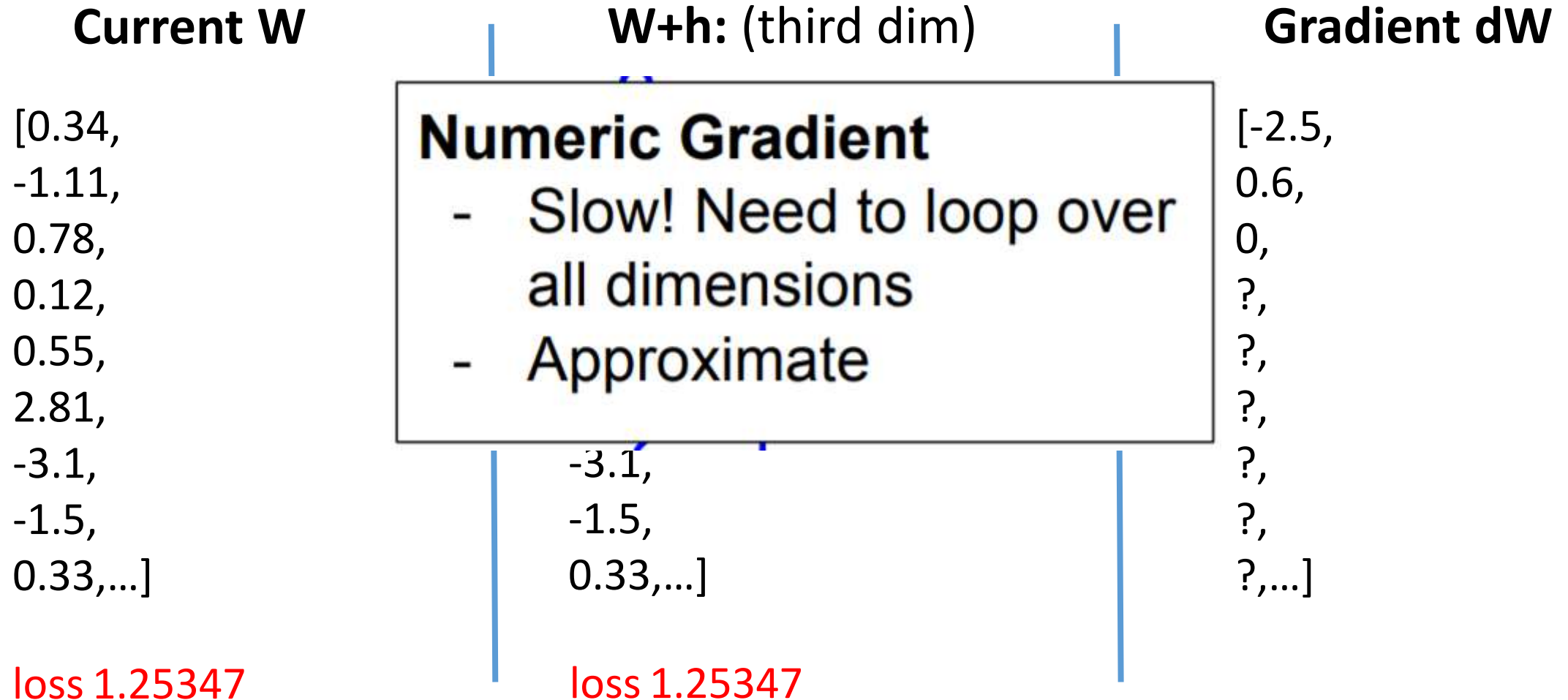
[-2.5,
0.6,
0,
?,
?,
?,
?,
?,
?,...]

$(1.25347 - 1.25347)/0.0001$
 $= 0$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?,...]

Optimization



Optimization

The loss is just a function of W :



[This image](#) is in the public domain



[This image](#) is in the public domain

Optimization

The loss is just a function of W :

Use calculus to compute an analytic gradient



[This image](#) is in the public domain



[This image](#) is in the public domain

Optimization

Current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

**dW (some
function)**



Gradient dW

[-2.5,
0.6,
0,
0.2,
0.7,
-0.5,
1.1,
1.3,
-2.1, ...]

Optimization

- Numerical gradient: **approximate**, **slow**, **easy to write**

Optimization

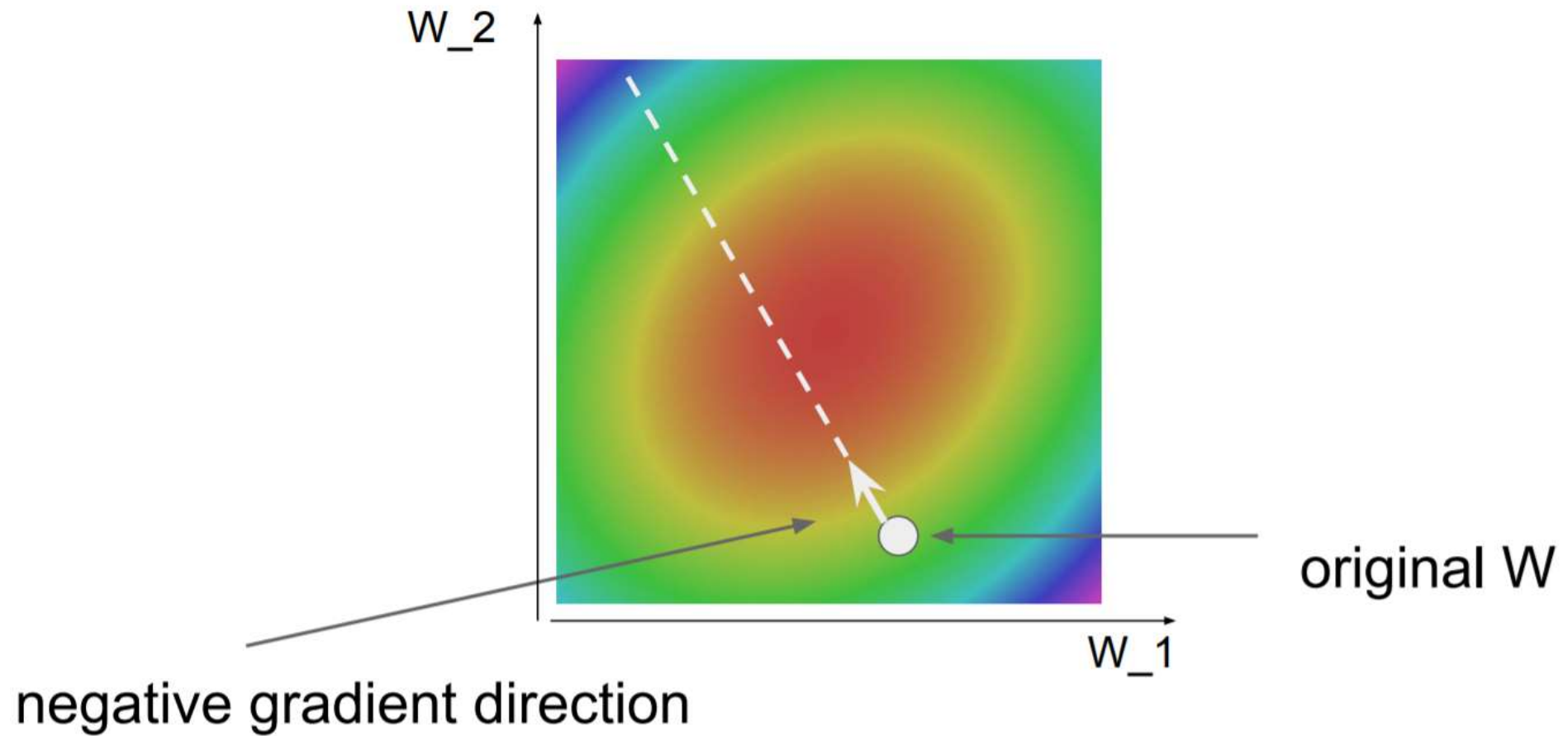
- Numerical gradient: approximate, slow, easy to write
- Analytic gradient: exact, fast, error-prone

Optimization

- Numerical gradient: **approximate**, **slow**, **easy to write**
- Analytic gradient: **exact**, **fast**, **error-prone**

In practice: **Always** use analytic gradient, but check implementation with numerical gradient. This is called a **gradient check**.

Gradient Descent



Linear regression (Optimization)

- Given a training set, how do we learn, the parameters θ ?

Linear regression (Optimization)

- Given a training set, how do we learn, the parameters θ ?
- We define the cost function:

Linear regression (Optimization)

- Given a training set, how do we learn, the parameters θ ?
- We define the cost function:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Linear regression (Optimization)

- Given a training set, how do we learn, the parameters θ ?
- We define the cost function:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Analitical solution

$$\theta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Linear regression (Optimization)

- Given a training set, how do we learn, the parameters θ ?
- We define the cost function:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Analitical solution

$$\theta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Numerical solution

?

Linear regression (Optimization)

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Cost function

Linear regression (Optimization)

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\sum_{i=0}^n \theta_i x_i$$

Cost function

Linear regression (Optimization)

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$


Cost function

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad j = 0, \dots, n$$

Gradient descent

Linear regression (Optimization)

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$


Cost function

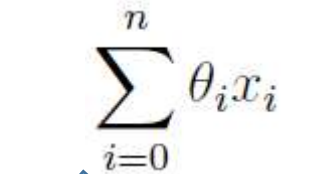
initial guess



$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad j = 0, \dots, n$$

Gradient descent

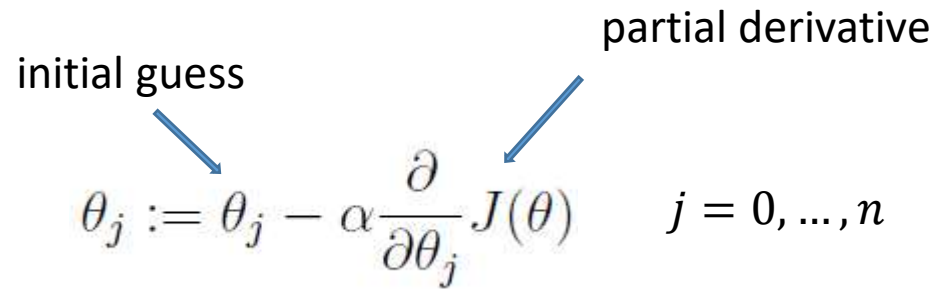
Linear regression (Optimization)

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$


Cost function

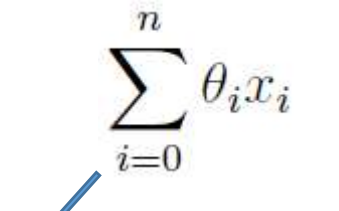
initial guess

partial derivative

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad j = 0, \dots, n$$


Gradient descent

Linear regression (Optimization)

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$


$\sum_{i=0}^n \theta_i x_i$

Cost function

initial guess

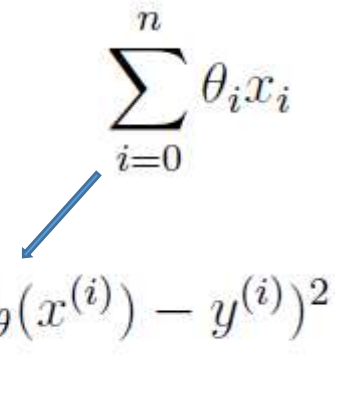
partial derivative

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad j = 0, \dots, n$$

learning rate

Gradient descent

Linear regression (Optimization)

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$


Cost function

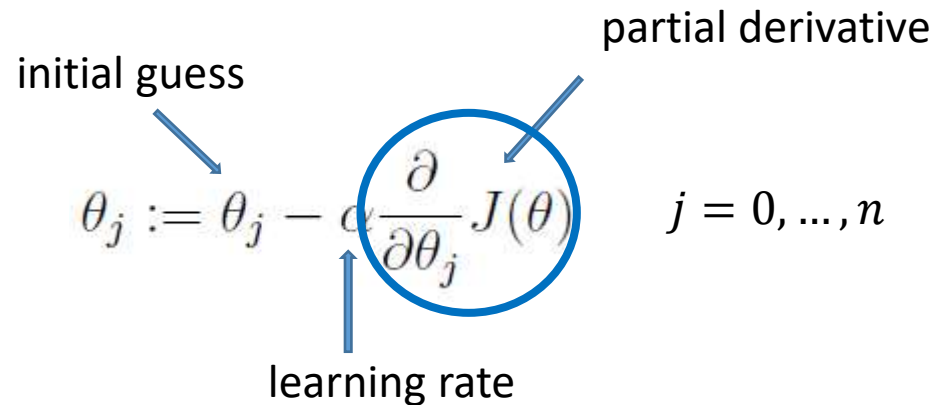
initial guess

partial derivative

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

learning rate

$j = 0, \dots, n$



Gradient descent

Linear regression (Optimization)

If we have only one training example (x, y) , we can neglect the sum in the definition of J .

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2$$

Linear regression (Optimization)

If we have only one training example (x, y) , we can neglect the sum in the definition of J .

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2$$

Chain rule: $(f \circ g)'(t) = f'(g(t)) \cdot g'(t)$

Linear regression (Optimization)

If we have only one training example (x, y) , we can neglect the sum in the definition of J .

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 \\ \text{Chain rule: } (f \circ g)'(t) &= f'(g(t)) \cdot g'(t) \end{aligned}$$
$$= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y)$$

Linear regression (Optimization)

If we have only one training example (x, y) , we can neglect the sum in the definition of J .

$$\begin{aligned}\frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 \\ &= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y)\end{aligned}$$

Chain rule: $(f \circ g)'(t) = f'(g(t)) \cdot g'(t)$

$$h_{\theta}(x) = \sum_{i=0}^n \theta_i x_i$$

Linear regression (Optimization)

If we have only one training example (x, y) , we can neglect the sum in the definition of J .

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 \\ \text{Chain rule: } (f \circ g)'(t) &= f'(g(t)) \cdot g'(t) &&= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y) \\ h_{\theta}(x) &= \sum_{i=0}^n \theta_i x_i &&= (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^n \theta_i x_i - y \right) \end{aligned}$$

Linear regression (Optimization)

If we have only one training example (x, y) , we can neglect the sum in the definition of J .

$$\begin{aligned}\frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 \\&= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y) \\&= (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^n \theta_i x_i - y \right)\end{aligned}$$

Chain rule: $(f \circ g)'(t) = f'(g(t)) \cdot g'(t)$

$$h_{\theta}(x) = \sum_{i=0}^n \theta_i x_i$$
$$\frac{\partial}{\partial \theta_j} (\theta_0 x_0 + \theta_1 x_1 + \cdots + \theta_n x_n) = x_j$$

Linear regression (Optimization)

If we have only one training example (x, y) , we can neglect the sum in the definition of J .

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 \\ &= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y) \\ &= (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^n \theta_i x_i - y \right) \\ &= (h_{\theta}(x) - y) x_j \end{aligned}$$

Chain rule: $(f \circ g)'(t) = f'(g(t)) \cdot g'(t)$

$$h_{\theta}(x) = \sum_{i=0}^n \theta_i x_i$$
$$\frac{\partial}{\partial \theta_j} (\theta_0 x_0 + \theta_1 x_1 + \cdots + \theta_n x_n) = x_j$$

Linear regression (Optimization)

If we have only one training example (x, y) , we can neglect the sum in the definition of J .

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 \\ &= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y) \\ &= (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^n \theta_i x_i - y \right) \\ &= (h_{\theta}(x) - y) x_j \end{aligned}$$

Chain rule: $(f \circ g)'(t) = f'(g(t)) \cdot g'(t)$

$$h_{\theta}(x) = \sum_{i=0}^n \theta_i x_i$$
$$\frac{\partial}{\partial \theta_j} (\theta_0 x_0 + \theta_1 x_1 + \dots + \theta_n x_n) = x_j$$

For a single training example, this gives the update rule:

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$$

Linear regression (Optimization)

Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)} \quad (\text{for every } j)$$

}

Batch gradient descent

This method looks at every example in the entire training set on every step

Linear regression (Optimization)

Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)} \quad (\text{for every } j)$$

}

Batch gradient descent

This method looks at every example in the entire training set on every step

Loop {

for i=1 to m, {

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)} \quad (\text{for every } j)$$

}

}

Stochastic gradient descent

This method repeatedly runs through the training set, and each time it encounters a training example, it updates the parameters.

Linear regression (Optimization)

Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)} \quad (\text{for every } j)$$

}

Batch gradient descent

This method looks at every example in the entire training set on every step

Loop {

for i=1 to m, {

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)} \quad (\text{for every } j)$$

}

}

Stochastic gradient descent

This method repeatedly runs through the training set, and each time it encounters a training example, it updates the parameters.

Often, stochastic gradient descent gets θ “close” to the minimum much faster than batch gradient descent.

Logistic Regression (Optimization)

Cost function

$$\ell(\theta) = -\sum_{i=1}^m y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))$$

Logistic Regression (Optimization)

Cost function

$$\ell(\theta) = -\sum_{i=1}^m y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))$$

hypothesis

$$h(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

Logistic Regression (Optimization)

Cost function

$$\ell(\theta) = - \sum_{i=1}^m y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))$$

hypothesis

$$h(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

Working with just one training example (x, y)

$$\frac{\partial}{\partial \theta_j} \ell(\theta) = - \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) \frac{\partial}{\partial \theta_j} g(\theta^T x)$$

Logistic Regression (Optimization)

Cost function

$$\ell(\theta) = - \sum_{i=1}^m y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))$$

hypothesis

$$h(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

Working with just one training example (x, y)

$$\begin{aligned} \frac{\partial}{\partial \theta_j} \ell(\theta) &= - \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) \frac{\partial}{\partial \theta_j} g(\theta^T x) \\ &= - \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) g(\theta^T x)(1 - g(\theta^T x)) \frac{\partial}{\partial \theta_j} \theta^T x \end{aligned}$$

Logistic Regression (Optimization)

Cost function

$$\ell(\theta) = -\sum_{i=1}^m y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))$$

hypothesis

$$h(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

Working with just one training example (x, y)

$$\begin{aligned} \frac{\partial}{\partial \theta_j} \ell(\theta) &= - \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) \frac{\partial}{\partial \theta_j} g(\theta^T x) \\ &= - \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) g(\theta^T x)(1 - g(\theta^T x)) \frac{\partial}{\partial \theta_j} \theta^T x \\ &= - (y(1 - g(\theta^T x)) - (1 - y)g(\theta^T x)) x_j \end{aligned}$$

Logistic Regression (Optimization)

Cost function

$$\ell(\theta) = - \sum_{i=1}^m y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))$$

hypothesis

$$h(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

Working with just one training example (x, y)

$$\begin{aligned} \frac{\partial}{\partial \theta_j} \ell(\theta) &= - \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) \frac{\partial}{\partial \theta_j} g(\theta^T x) \\ &= - \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) g(\theta^T x)(1 - g(\theta^T x)) \frac{\partial}{\partial \theta_j} \theta^T x \\ &= - (y(1 - g(\theta^T x)) - (1 - y)g(\theta^T x)) x_j \\ &= - (y - h_{\theta}(x)) x_j \end{aligned}$$

Logistic Regression (Optimization)

Cost function

$$\ell(\theta) = - \sum_{i=1}^m y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))$$

hypothesis

$$h(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

Working with just one training example (x, y)

$$\begin{aligned} \frac{\partial}{\partial \theta_j} \ell(\theta) &= - \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) \frac{\partial}{\partial \theta_j} g(\theta^T x) \\ &= - \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) g(\theta^T x)(1 - g(\theta^T x)) \frac{\partial}{\partial \theta_j} \theta^T x \\ &= - (y(1 - g(\theta^T x)) - (1 - y)g(\theta^T x)) x_j \\ &= - (y - h_{\theta}(x)) x_j \\ &= (h_{\theta}(x) - y) x_j \end{aligned}$$